

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ

FACULTAD DE CIENCIAS E INGENIERÍA

PROGRAMACIÓN 3

6ta práctica (tipo b)

(Primer Semestre 2026)

Indicaciones Generales:

- Duración 1h 50 minutos (Inicio: 8:00 a.m. - Fin: 9:50 a.m.) (Subida y verificación de archivos en PAIDEIA: 9:50 a.m. a 9:55 a.m.) (Salida del Laboratorio: 9:55 a.m. a 10:00 a.m.)
- Se les recuerda que, de acuerdo con el reglamento disciplinario de nuestra institución, constituye una falta grave copiar del trabajo realizado por otro estudiante o cometer plagio para el desarrollo de esta práctica.
- Para el desarrollo de toda la práctica debe utilizar el sistema operativo Windows y Java 25.
- Para el desarrollo de las preguntas debe usar IntelliJ.
- Para el desarrollo de la práctica puede usar sentencias SQL o Procedimientos Almacenados.
- Está permitido el uso de Internet (únicamente para consultar páginas oficiales de Oracle y Microsoft). No obstante, está prohibida toda forma de comunicación con otros estudiantes o terceros.
- PUEDE UTILIZAR MATERIAL DE CONSULTA. Antes de comenzar el laboratorio, descargue todos los proyectos, apuntes, diapositivas que utilizará.
- Se considerará en la calificación el uso de buenas prácticas de programación (aquellas vistas en clase).
- Al finalizar el laboratorio debe entregar un archivo llamado credenciales.txt con el nombre del servidor, base de datos, usuario y contraseña de su base de datos.

Puntaje total: 20 puntos

Cuestionario

- La finalidad principal de este laboratorio es la de reforzar los conceptos contenidos en las clases 06, 07, 08, 09 y 10: Conexión a Bases de Datos.
- Al finalizar la práctica, comprima la carpeta dada en las indicaciones iniciales empleando el programa Zip que viene por defecto en el Windows, no se aceptarán los trabajos compactados con otros programas como RAR, WinRAR, 7zip o similares.

Descripción del caso

Una institución educativa desea implementar un sistema piloto de evaluación llamado **TestSoft** que consta de dos partes. La primera parte corresponde a un banco de preguntas académicas registradas en el sistema, desde donde se seleccionan automáticamente preguntas para conformar exámenes personalizados para un conjunto de alumnos. Dichos exámenes registran información relevante como: Código del alumno, nombre del alumno, título del examen, fecha de creación y las preguntas asignadas al examen.

El objetivo principal es generar automáticamente exámenes para los alumnos registrados, asignando a cada uno un conjunto de 10 preguntas seleccionadas desde el banco disponible. Una vez generados, los datos deben ser almacenados para su posterior consulta y visualización.

En otras palabras, se requiere que el sistema piloto no solo actúe como un mecanismo de generación automática de exámenes, sino también como una herramienta que alimente automáticamente la base de datos de evaluaciones para una mejor gestión del registro de exámenes.

En resumen, la institución educativa requiere un sistema que permita:

1. Leer un banco de preguntas disponible para la generación de evaluaciones.
2. Leer la información de los alumnos registrados en el sistema.
3. Seleccionar aleatoriamente un conjunto de 10 preguntas por cada alumno y generar un examen.
4. Generar un listado estructurado de cada examen que incluya: alumno asignado, título del examen, fecha de creación y listado de preguntas seleccionadas.

Tenga en cuenta que el paquete base de la solución será: `pe.edu.pucp.testsoft`

Base de datos TestSoft

A continuación, se muestra el modelo de base de datos del sistema de **TestSoft**. Se registran los alumnos (código, nombre, correo) que participarán en el proceso de evaluación. Asimismo, se registra un banco de preguntas, donde cada pregunta almacena únicamente su enunciado. Los exámenes generados se asocian a un alumno y almacenan información como el título y la fecha de creación. La relación entre exámenes y preguntas se maneja mediante la tabla intermedia examen_pregunta, la cual permite consolidar qué preguntas fueron asignadas a cada examen y en qué orden aparecen dentro del mismo.

El diagrama ER se muestra en la Figura 1.

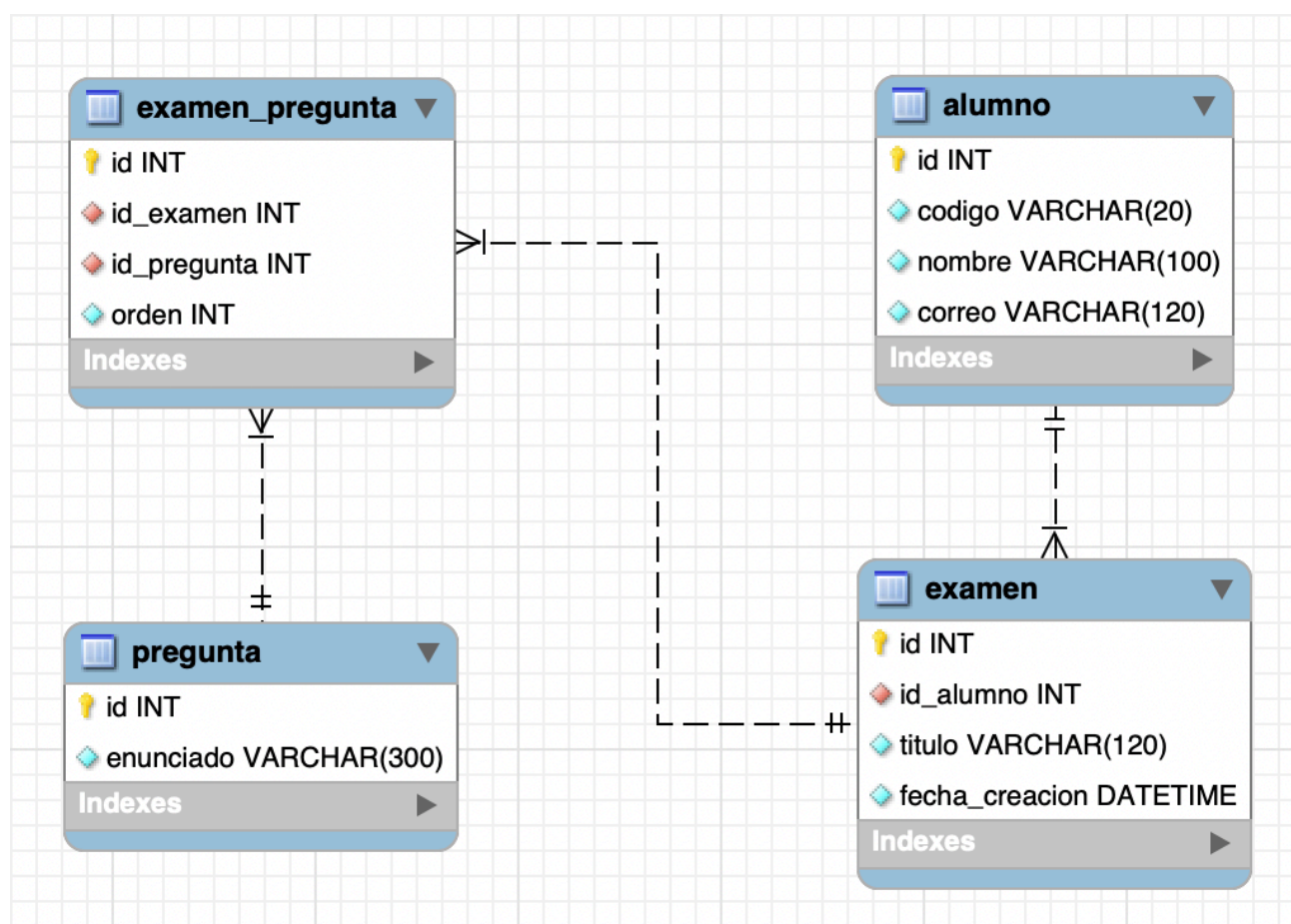


Figura 1 - Diagrama ER de TestSoft

Se proporcionará un script SQL para la creación de la base de datos en MySQL y **la inicialización de la base de datos en MySQL, incluyendo el registro de alumnos y el banco de preguntas.** También se proporcionará un script SQL con los procedimientos almacenados necesarios para el laboratorio. Usted puede usar y/o crear procedimientos o sentencias SQL según crea conveniente. **El script deberá ser ejecutado antes de iniciar con el desarrollo del laboratorio.**

Pregunta 01: Creación de los módulos Maven y configuración de sus dependencias iniciales (5 puntos)

1. Creación de la solución en Java basada en Maven con arquitectura en capas:
El sistema debe seguir una arquitectura de capas desacoplada, estructurada en los siguientes cinco módulos: **(2 puntos)**
 - a. Modelo: Definición de entidades de dominio (**TestSoftModelo**).
 - b. DB Manager: Gestión de la conexión y configuración de la base de datos (**TestSoftDBManager**).
 - c. Persistencia: Implementación de DAOs y acceso a datos (**TestSoftPersistencia**).
 - d. Negocio: Lógica de procesos, transacciones, validaciones y reglas del sistema (**TestSoftNegocio**).
 - e. Aplicación: Punto de entrada, controladores e interfaz de usuario (**TestSoftApp**).

Los nombres de los módulos proporcionados en este documento son referenciales. Para la entrega final, es obligatorio sustituirlos por el esquema de nombrado usado en su turno del curso.
2. Configurar las dependencias necesarias para cada módulo, se evaluará la correcta asignación de dependencias: **(1 punto)**
 - a. Dependencias entre módulos.
 - b. Dependencia al driver de MySQL.
3. Implementar un DBManager con soporte para MySQL. **(1 punto)**
4. Implementar la capa de entidades de dominio. **(1 punto)**

Pregunta 02: Implementación de la capa de persistencia (8 puntos)

Implemente la capa de persistencia del sistema, respetando la arquitectura propuesta. Esta capa debe permitir recuperar la información necesaria para la generación de exámenes, así como registrar y consultar los exámenes creados por el sistema.

Para esta pregunta, se solicita desarrollar únicamente los DAOs necesarios para atender los siguientes requerimientos:

1. Obtener el listado completo de alumnos registrados en la base de datos. **(1 punto)**
2. Obtener el listado completo de preguntas disponibles en el banco de preguntas. **(1 punto)**
3. Registrar un nuevo examen asociado a un alumno. **(4 puntos)**
 - a. Registrar la relación entre un examen y cada una de sus preguntas asignadas.
4. Recuperar el listado de exámenes generados, incluyendo su información principal y las preguntas asociadas a cada uno. **(2 puntos)**

No se requiere implementar operaciones adicionales distintas a las expresamente indicadas. La evaluación considerará la correcta aplicación del patrón DAO y la correspondencia entre sus métodos y las operaciones de persistencia requeridas.

Pregunta 03: Implementación de la capa de negocio (7 puntos)

El módulo de la aplicación (e.j, TestSoftApp) contendrá dos clases con sus respectivos métodos main

1. CrearExámenes
2. MostrarExámenes

CrearExámenes.java

```
public class CrearExámenes {
    public static void main(String[] args) {
        AlumnoBO alumnoBO = new AlumnoBOImpl();
        ExamenBO examenBO = new ExamenBOImpl();
        PreguntaBO preguntaBO = new PreguntaBOImpl();

        List<Alumno> alumnos = alumnoBO.leerTodos();

        for (Alumno alumno : alumnos) {
            List<Pregunta> preguntasSeleccionadas =
                preguntaBO.seleccionarPreguntasAleatorias();
            examenBO.crearExamenConPreguntas(
                alumno,
                "Examen configurado para " + alumno.getCodigo(),
                preguntasSeleccionadas
            );
        }

        System.out.println("Se generaron los exámenes.");
    }
}
```

MostrarExámenes.java

```
public class MostrarExámenes {
    public static void main(String[] args) {
        ExamenBO examenBO = new ExamenBOImpl();
        List<Examen> examenes = examenBO.leerTodos();

        for (Examen examen : examenes) {
            System.out.println("-----");
            System.out.println("Examen ID: " + examen.getId());
            if (examen.getFechaCreacion() != null) {
                System.out.println("Fecha de creación: " + examen.getFechaCreacion());
            }
            System.out.println("Alumno: " + examen.getAlumno().getCodigo() + " - " +
examen.getAlumno().getNombre());
            System.out.println("Título: " + examen.getTitulo());
            System.out.println("Preguntas asignadas: " + examen.getPreguntas().size());
            for (int i = 0; i < examen.getPreguntas().size(); i++) {
                Pregunta pregunta = examen.getPreguntas().get(i);
                System.out.println((i + 1) + ". " + pregunta.getEnunciado());
            }
        }
    }
}
```

Los nombres ExamenBO, AlumnoBO, ExamenBOImpl y AlumnoBOImpl son referenciales, debe usar la convención de nombrado usado en su turno del curso.

Implemente la capa de negocio del sistema, encargada de coordinar las operaciones requeridas para la generación y consulta de exámenes. Esta capa debe actuar como intermediaria entre la aplicación y la persistencia, centralizando las reglas del sistema y asegurando que las operaciones se ejecuten de manera consistente.

Para esta pregunta, se solicita desarrollar únicamente los BOs necesarios para cubrir los siguientes requerimientos funcionales:

Crear Exámenes: (5 puntos)

1. Recuperar el listado de alumnos disponibles para la asignación de exámenes (método **leerTodos** del objeto de negocio de alumnos). **(1 punto)**
2. Recuperar el listado de preguntas disponibles en el banco de preguntas (método **seleccionarPreguntasAleatorias** del objeto de negocio de preguntas). **(1 punto)**
 - a. Este método deberá usar el DAO de Preguntas para obtener todo el banco de preguntas.
 - b. Posteriormente deberá generar una lista aleatoria de 10 preguntas no repetidas.
 - c. Tip: Puede usar `Collections.shuffle(copia)`, como se muestra a continuación.

```
List<Pregunta> copia = new ArrayList<>(bancoPreguntas);
Collections.shuffle(copia);
return new ArrayList<>(copia.subList(0, 10));
```

3. Crear exámenes garantizando que el registro del examen y de sus preguntas asociadas se ejecute como una única unidad lógica de trabajo, manteniendo la consistencia de la información. En caso de error en cualquiera de las operaciones, no deberá persistir ningún cambio. Esta lógica deberá implementarse en el método **crearExamenConPreguntas** del objeto de negocio de exámenes.. **(3 puntos)**

Mostrar Exámenes: (2 puntos)

1. Recuperar los exámenes registrados para su posterior visualización en la aplicación (método **leerTodos** del objeto de negocio de exámenes).

Profesores del curso: Freddy Paz
David Allasi

José Corcuera
Eric Huiza

Pando, 29 de abril de 2026